

Architektura systemów baz danych

The architecture of database systems

dr hab. inż. Maciej Grzenda

M.Grzenda@mini.pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund

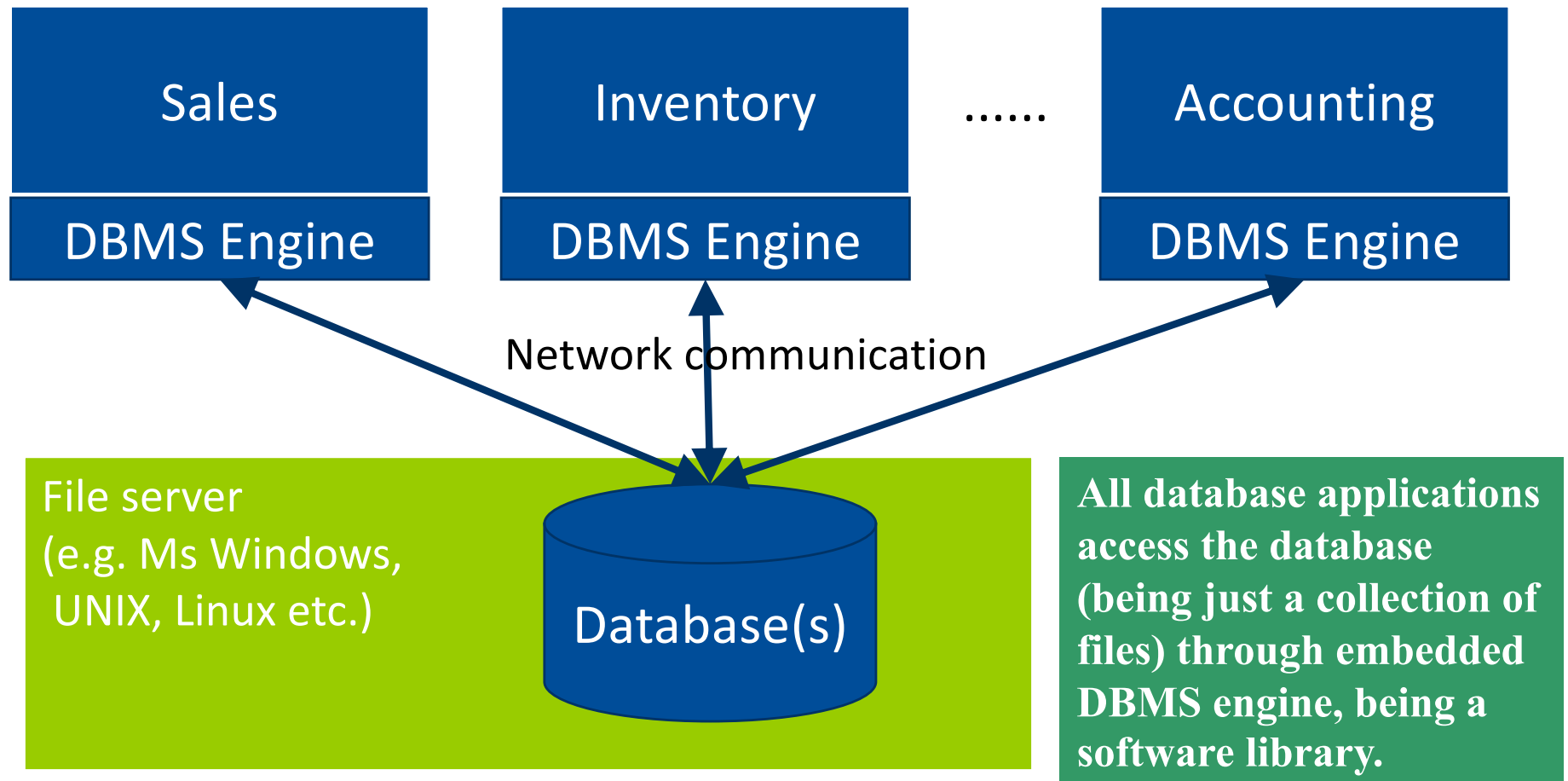


MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Objectives

- Frequently more than one instance of a DBMS exists in one organisation
- This is for many reasons such as:
 - Different database applications require different DBMS platforms or different versions of these platforms
 - Key enterprise systems may run on their own instances to increase stability and simplify system management
 - Standby servers may be used to ensure high availability of the systems
- Hence, how multiple instances of DBMS platforms are managed in terms of data management is a vital aspect of data management

A bit of history: File-based approach



Why not file-based approach for multi-user systems?

- Applications access the database by using embedded libraries, thus:
 - The data is typically transferred over the network to be updated and queried on the users' workstations,
 - Poor scalability of the solution is observed when databases get larger due to:
 - Possible network congestion,
 - Increased demand for CPU time on client workstations since data processing occurs on client workstations,
 - Limited security of the data:
 - usu. data files can be easily copied and/or modified by the users
 - Network problems may result in partial updates and/or damaged file structure.
- While file-based approach can still be observed for some small-scale systems, we will focus on more demanding (and complex) settings in this lecture

When to use file-based i.e. serverless approach?

- File-based approach, i.e. serverless databases can be used for specific use cases, when a local lightweight database solution is needed. This includes running databases on:
 - Mobile phones,
 - Cameras,
 - Vehicles,
 - Remote sensors,
 - Drones,
 - Medical devices and other Internet of Things devices,
 - But also to enable local data analysis, e.g. in JupyterLab
- A very popular serverless solution is SQLite (<https://sqlite.org>), which is an in-process library that can be used to manage local data as a relational database, query and modify it using SQL. All the source code of SQLite is defined with one file in C language.
- The size of the library is typically less than 1MB.

Note that typically, only user will access a serverless database at a time. Still, the database library can offer transactional processing.

SQLite examples

```
import sqlite3
conn = sqlite3.connect("/Users/.../my_database.db")
df = pd.read_sql_query("select count(*),Autor_ID from \
                        (select distinct Job,Autor_ID from papers) group by \
                        Autor_Id \
                        having count(*)>1", conn)
df
```

Rather than connecting to a service, we open a file

```
cursor = conn.cursor()
cursor.execute("update grants set Department=TRIM(\
                SUBSTR(\
                    Project_Manager,\
                    INSTR(Project_Manager, '(')+1,\
                    INSTR(Project_Manager, ')')-1 -\
INSTR(Project_Manager, '(') \
                )\
                );")
conn.commit()
```

A dataframe can be created out of SQL result set

Transactional processing is supported

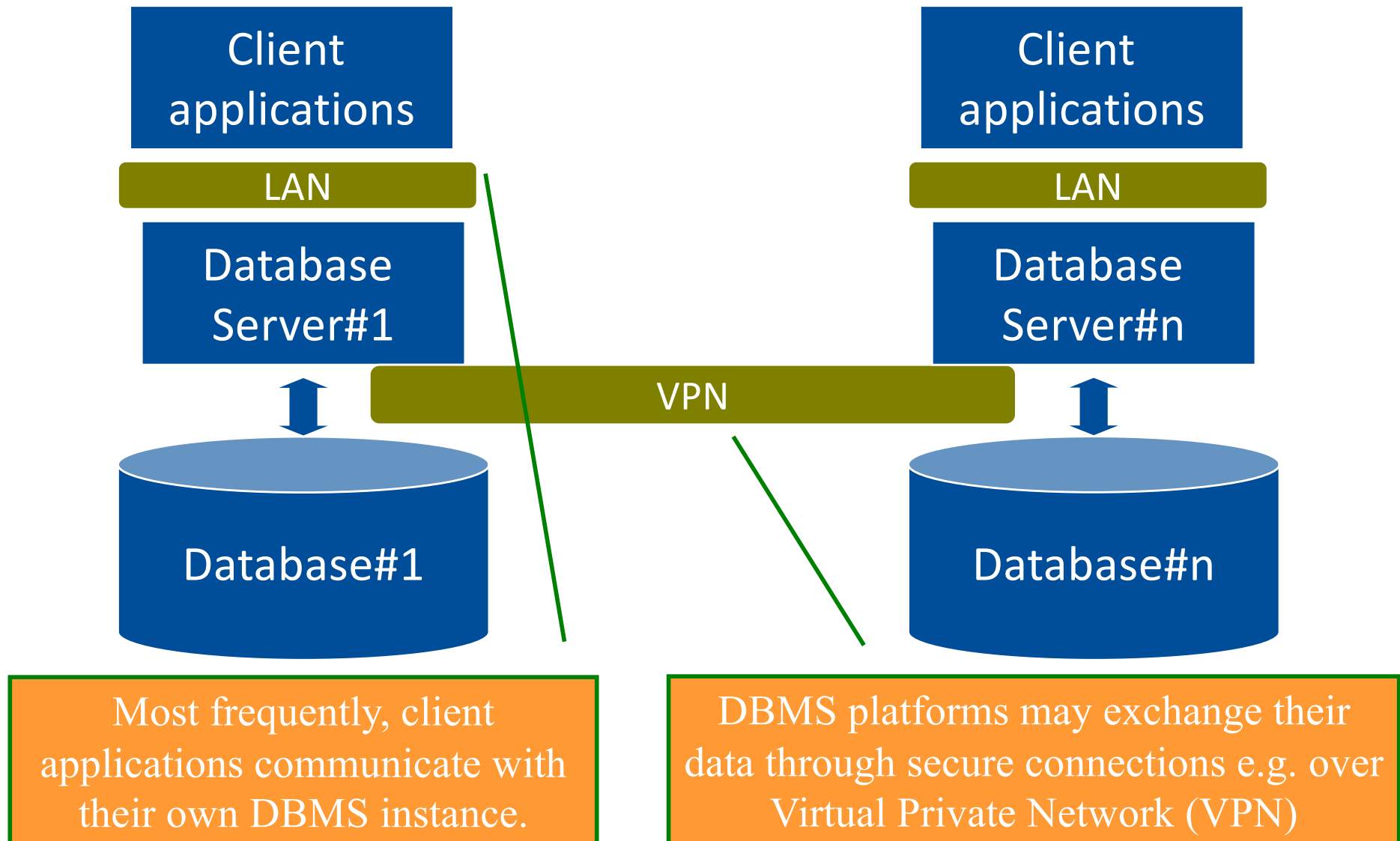
A sample tool to manage different databases including SQLite is Dbeaver
<https://dbeaver.io>

Distributed databases

Rozproszone bazy danych

- Distributed databases are databases composed of multiple databases
- Example: country-wide tax database can be a sum of regional databases
- Distributed databases are used:
 - For mission-critical systems - when stable connection to the remote server can not be provided local database may be added
 - To decrease response time
 - To provide for autonomous maintenance of software systems
 - When different software systems in an organisation require their own DBMS platforms (heterogeneous distributed databases)
 - When most of the time access to local database only is sufficient

Distributed databases – sample architecture



Distributed transactions

Transakcje rozproszone

- Distributed transactions (DTs) are transactions that require multiple databases or multiple DBMSs to participate in
- If every database has its own private transaction log, then even a transaction that affects two different databases on the same database server requires a distributed transaction
- To execute a distributed transaction means to ensure ACID properties, which is more difficult in a distributed setting

What is a distributed transaction?

- DT requires significant overhead: typically two phase commit protocol (*dwufazowy protokół potwierdzeń*) is involved. This protocol relies on two phases:
 - **Prepare phase**: voting made by all involved DBMS instances
 - **Commit phase**: distribution of decision to all involved DBMS instances, which have to apply commit to their part of overall changes or roll back their part of the changes
- A coordinating service is required to manage the two stages. In the case of Ms SQL Server, such a service is named Microsoft Distributed Transaction Coordinator
- Notice that the following problems must be addressed:
 - Some of participating DBMSs may be not available,
 - Some of DBMSs may not accept requested changes e.g. due to constraint violation

Distributed transaction – an example

This batch of SQL statements is submitted to one of participating servers

```
DELETE FROM  
KRAKOW_OFFICE.Sales.dbo.Customers  
WHERE CustomerId='1020'  
  
INSERT INTO  
POZNAN_OFFICE.Sales.dbo.Customers  
VALUES ('1020', 'Top Company', ...)
```

Client
applications

LAN

KRAKOW_OFFICE



Database

VPN

Client
application

LAN

POZNAN_OFFICE



Database

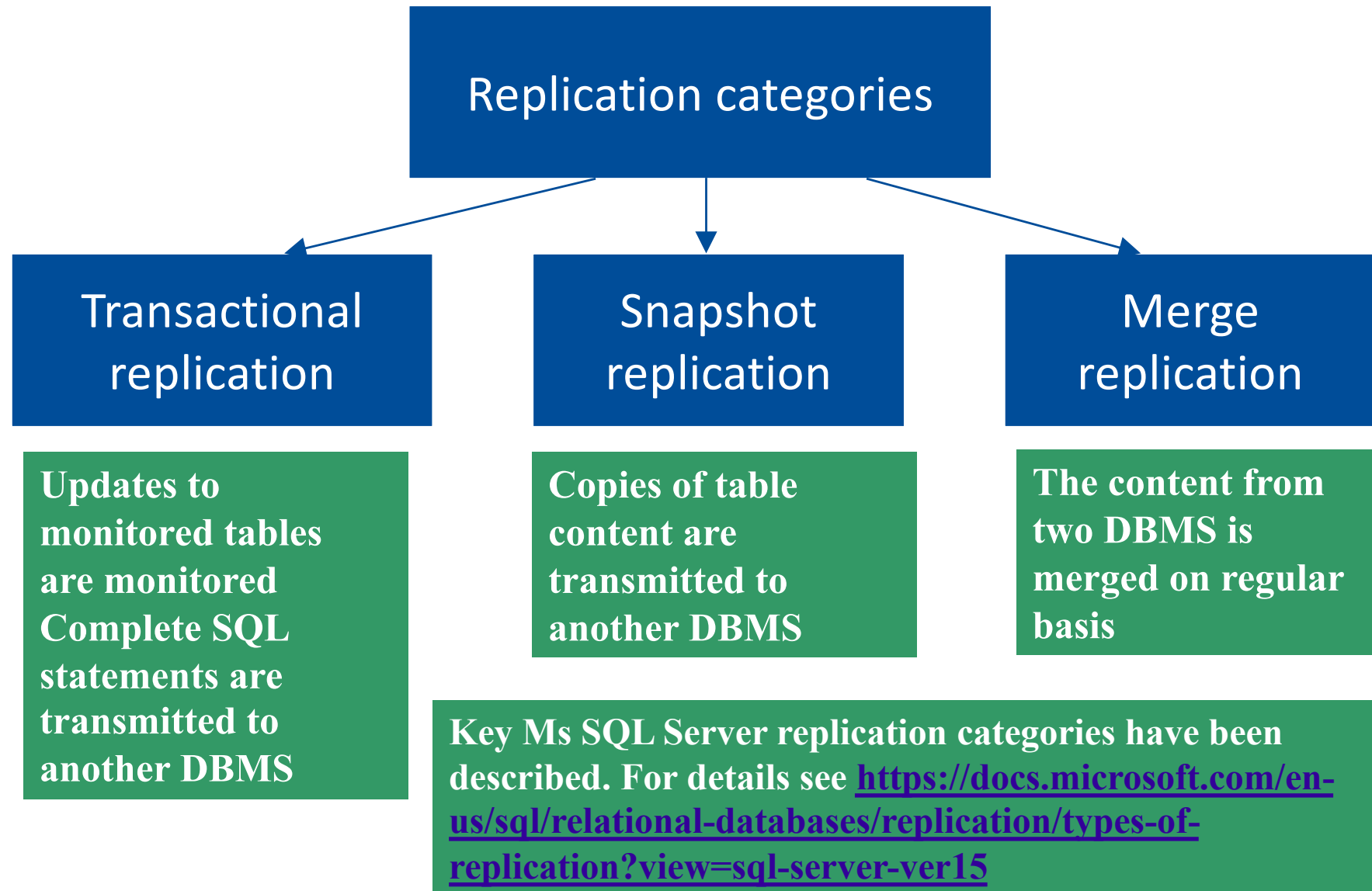
Example - discussion

- A DT may be submitted to one of participating DBMS servers
- In the case of Ms SQL Server, it starts with:
`BEGIN DISTRIBUTED { TRAN | TRANSACTION }`
- The transaction coordinator must communicate with both DBMSs to:
 - Confirm updates are possible e.g. there is no 1025 customer in POZNAN_OFFICE DBMS
 - Transactionally execute updates to both servers

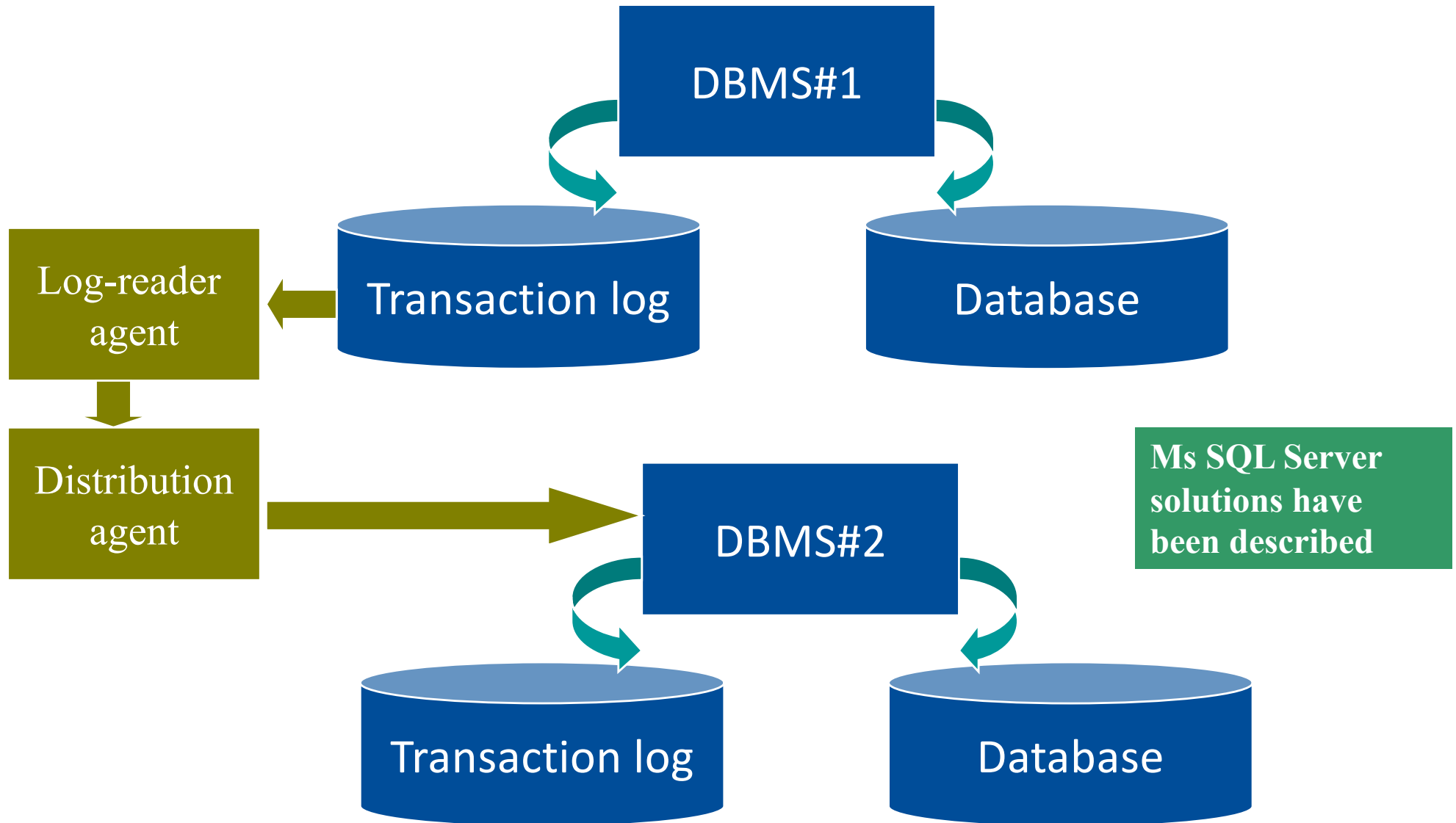
Data synchronisation in distributed databases

Method	Short description	Remarks
Distributed transactions	• Updates are made to all participating servers at the same time • Transaction processing is stopped whenever any of the servers is not available	• The only method that guarantees on-line consistency of participating databases • Major processing overhead results in reduced performance
Replication	• The updates to monitored tables are transferred to other participating servers. • Can be used to constantly update the data in stand-by servers.	• Accepts latency in data transfer • Updates to participating databases are postponed, thus the system as a whole may accept user's input even if one of the servers does not respond

Replication overview



Transactional replication



Transactional replication

- Can be treated as a kind of add-on functionality to the core of DBMS:
 - Log reader agent monitors on-line all the changes to some of the tables (say: customers) by using transaction logs
 - It captures the changes to place them in a distribution database
 - Distribution agent delivers the SQL statements (namely INSERT, UPDATE and DELETE made on Customers table) to the subscriber periodically (e.g. every night) or on-line

Transactional replication – case study

- Task: the head office must have a complete list of invoices issued by a regional office by the end of the next business day. Both offices have their own DBMS instances.
- Solution #1:
 - Using distributed transactions, apply at the same time all the updates made on a regional server also on the central server
 - Drawback: no invoices can be issued once the central server gets unavailable
- Solution #2:
 - Set up transactional replication (also on-line replication)
 - In case the central server is not available, all the changes to invoices tables in a local database are still gathered by the log-reader agent and delivered later – once the connection between the servers is re-established

Snapshot and merge replication

- Snapshot replication:
 - Can be used to periodically replace the content of subscriber's table(s) with new content, without tracking individual changes
 - Example: publication of price list from the company's headquarter
- Merge replication:
 - Can be used to periodically merge the content of two different databases
 - Example: table of customers existing on the central server and mobile workstations of sales representatives. Central database is merged with new customers' data from mobile computers.

High availability (HA)

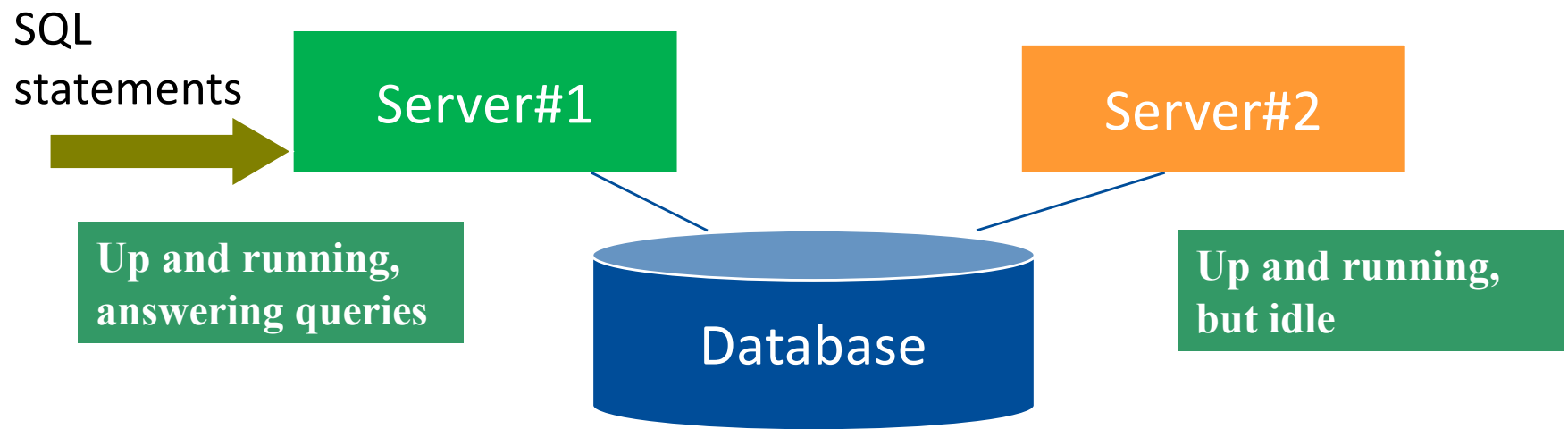
- In case of IT systems means that the system is available all the time (24x7)
- In practice, distinction between 99%, 99.9% and higher availability is made
- Typical HA architectures typically involve:
 - Redundant devices:
 - Redundant power supplies,
 - RAID (Redundant Array of Inexpensive Disks),
 - Multiple network interface cards,
 - Redundant network connections,
 - Clusters of servers.

HA: clusters

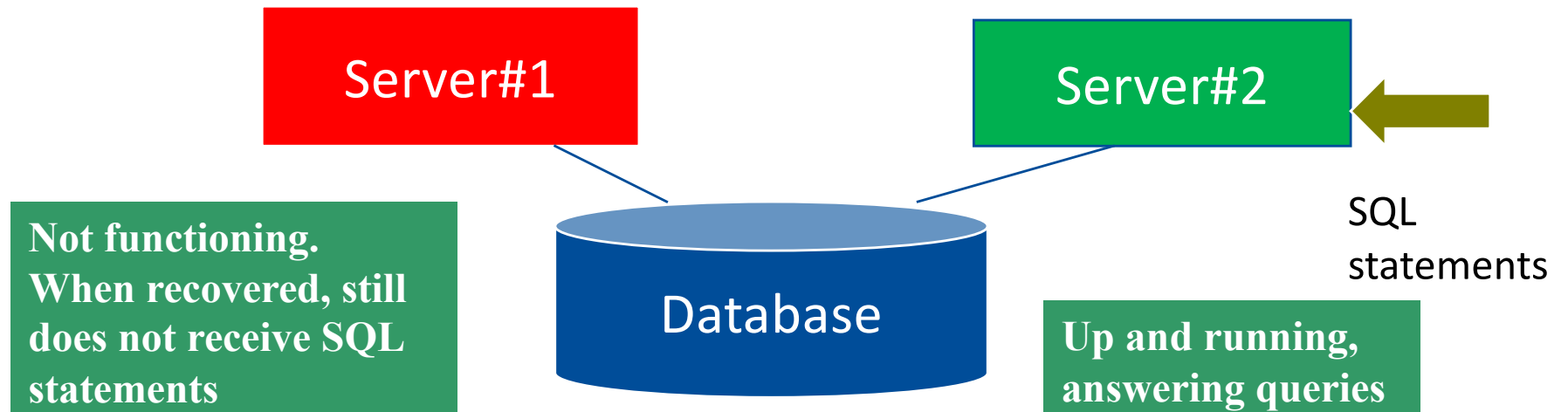
- Cluster: two or more similar interconnected servers usu. sharing the same set of disks (disk array)
- Different strategies exist. Examples:
 - Active/passive: one of the servers is ready to accept the requests, but normally passive. It plays the role of a standby server. Its role is to handle requests when the primary one fails
 - Active/active: both servers (or more than 2) share the load

Cold failover cluster

Before failover

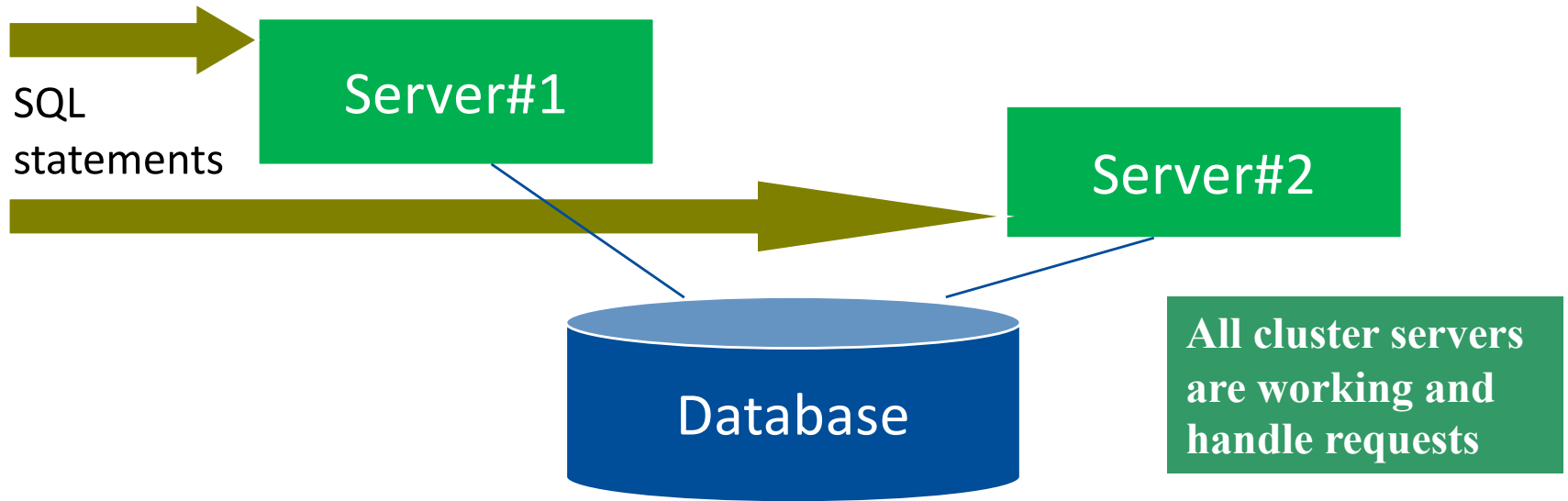


After failover

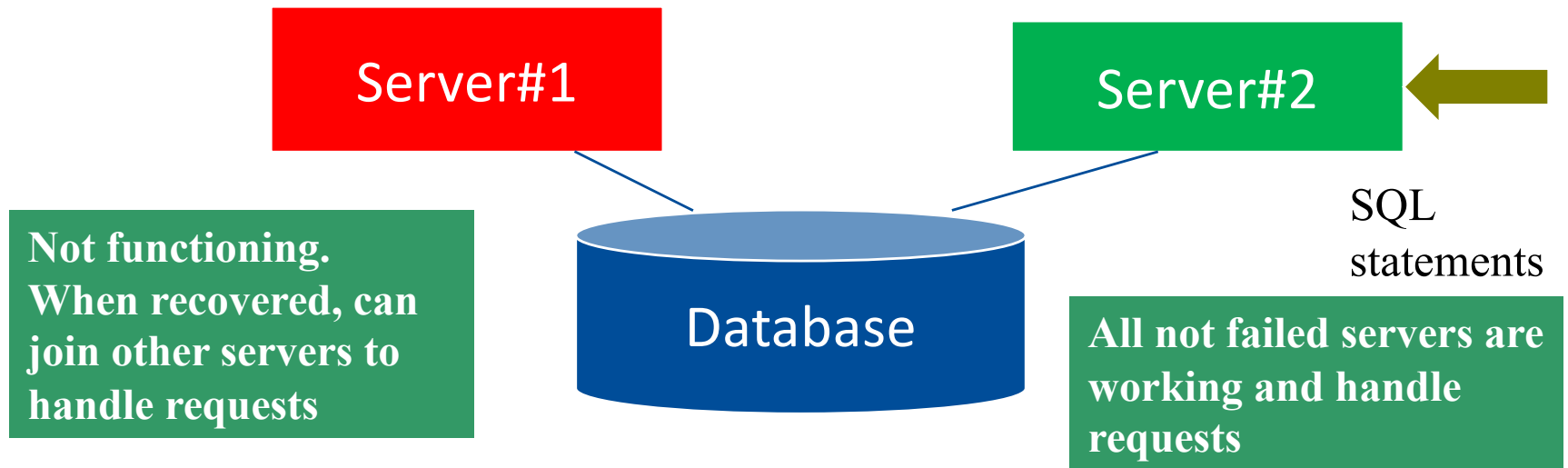


Hot failover cluster

Before failover



After failover



Summary

- Most frequently, multiple DBMS instances are used by an organisation
- This is because of:
 - Variety of database applications and DBMS platforms they need
 - Spatial distribution of organisations
 - High availability needs
- Most advanced platforms such as Oracle Real Applications Clusters (RAC) take advantage of a cluster of DBMS instances to combine high availability with high performance

Pytania ?

Zapraszam w trakcie wykładu
i w trakcie konsultacji

Miejsce i termin konsultacji
- na stronie:

[https://pages.mini.pw.edu.pl/
~grzendam/pl/dydaktyka.html](https://pages.mini.pw.edu.pl/~grzendam/pl/dydaktyka.html)



Questions?

I am looking forward to your questions during the lectures (including now) and during my office hours

For details on my office hours, please check my website:

https://pages.mini.pw.edu.pl/~grzendam/eng/dydaktyka_eng.html





**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.